

Athlete Intelligence — Incremental Migration Roadmap

Document type: Implementation Roadmap

Status: Awaiting approval — no code written

Strategy: Strangler-fig iterative migration. The application remains fully functional after every milestone. New systems run alongside old until proven stable. Nothing is deleted until its replacement has operated correctly for at least one complete cycle.

How to Read This Document

Each milestone entry contains:

- What changes — exact files and tables to be added
- What stays the same — what the ex

ist
in
g
sy
st
e
m
co
nti
nu
es
t
o
d
o
•
D
e
p
e
n
d
e
nc
ie
s
—
w
hi
ch
p
ri
or
m
ile
st
o
n
es
m
us
t
b
e
co
m
pl
et
e
fir
st
•
D
B
mi
gr
ati
o
n
ris
k
—
r
at
e
d
N
o
n

e
/
L
o
w
/
M
e
d
i
u
m
/
Hi
g
h,
w
ith
e
xp
la
n
ati
o
n
•
E
xi
sti
n
g
d
at
a
—
h
o
w
cu
rr
e
nt
a
thl
et
e
d
at
a
is
pr
es
er
ve
d
•
R
oll
b
ac
k
pl
a
n
—
e
xa
ct
st
e

ps
t
o
u
n
d
o
if
so
m
et
hi
n
g
g
o
es
w
ro
n
g

•
Ef
fo
rt

—
r
e
ali
sti
c
es
ti
m
at
e
fo
r
a
si
n
gl
e
e
n
gi
n
e
er

•
Te
st
g
at
e

—
w
h
at
m
us
t
p
as
s
b
ef
or
e

moving to the next milestone

Phase Overview

Phase	Name	Milestones	Total effort	Risk level
1	Critical Code Fixes	M1.1 – M1.7	2 days	None
2	Database Foundations	M2.1 – M2.3	1 day	Low
3	Source Registry	M3.1 – M3.3	2 days	Low
4	Population Status Tracking	M4.1 – M4.3	1 day	Low
5	Evidence Foundation	M5.1 – M5.4	5 days	Medium
6	Identity Engine	M6.1 – M6.4	4 days	Medium
7	Social Engine	M7.1 – M7.5	5 days	Medium
8	Job Queue	M8.1 – M8.4	5 days	Medium
9	Timeline Rebuild	M9.1 – M9.4	4 days	Medium
10	AI Layer Redesign	M10.1 – M10.4	10 days	High
11	Cutover & Cleanup	M11.1 – M11.3	5 days	Medium
Total		30 milestones	~44 days	

Data Preservation Guarantee

The following rule applies to every milestone in this roadmap:

No existing table is dropped, no existing column is removed, and no existing row is deleted until its replacement has been running in production for a minimum of 30 days and has been verified to contain equivalent or superior data.

Every schema change in Phases 1–9 is either:

- Additive (new table, new column)

— Zero risk to existing data

- Backfill (populate new structure from existing data)

—

existing database uncertainty handling guidelines, new structural population estimates in parallel

The cutover in Phase 11 is the only point at which old tables are deprecated, and only after a 30-day parallel-running period.

Phase 1 — Critical Code Fixes

Pure code changes. Zero schema changes. Zero migration risk. The application must be in better shape before any schema work begins.

M1.1 — Replace undocumented model alias ``gpt-5.6-luna``

What changes:

- artifacts/assembly-server/c/lib/autopop

ul
at
e.t
s:
C
h
a
n
g
e
m
o
d
el
fr
o
m
g
pt
-5
.6
-lu
n
a
to
g
pt
-4
o
fo
r
b
ot
h
th
e
di
sc
ov
er
y
ca
ll
a
n
d
th
e
str
uc
tu
re
d
ex
tr
ac
tio
n
ca
ll
•
ar
tif
ac
ts/
a
pi-
se
rv

er
/sr
c/l
ib/
p
h
ot
o-l
o
ok
u
p.t
s:
C
h
a
n
g
e
m
o
d
el
fr
o
m
g
pt
-5
.6
-lu
n
a
to
g
pt
-4
o-
mi
ni;
r
e
m
ov
e
re
as
o
ni
n
g
—
ef
fo
rt:
"
n
o
n
e"
p
ar
a
m
et
er
e
nti
rel

y
•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/s
u
m
m
ar
y.t
s:
C
h
a
n
g
e
m
o
d
el
fr
o
m
g
pt
-5
.6
-lu
n
a
to
g
pt
-4
o

What stays the same: All routes, schema, prompts, and logic are unchanged. Only the model string changes.

Dependencies: None. First milestone.

DB migration risk: None. No schema changes.

Existing data: Unaffected.

Rollback plan: Revert the three files to previous model strings. Single git revert.

Effort: 2 hours

Test gate:

Add a new test athlete via the UI. Confirm the dossier populates within 90 seconds.

Open the dossier Summary tab. Confirm the AI summary generates and streams.

Check server logs. Confirm no "model not found" or "unknown parameter" errors.

Check that gpt-5.6-luna no longer appears anywhere in the codebase (grep -r "gpt-5.6-luna").

M1.2 — Fix hardcoded date in research prompt

What changes:

•
ar
tif

ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
R
e
pl
ac
e
th
e
h
ar
dc
o
d
e
d
str
in
g
"T
o
d
ay
's
d
at
e
is
Ju
ly
2
9,
2
0
2
6.
"
wi
th

a
dy
n
a
mi
c
ex
pr
es
si
o

n
th
at
e
va
lu
at
es
a
t c
all
ti
m
e:
,
To
d
ay
's
d
at
e
is
\${
n
e
w
D
at
e(
)
.t
ol
S
O
St
ri
n
g(
)
.s
pli
t(
T'
)
[
0]
}
.

What stays the same: Everything else in the prompt and pipeline.

Dependencies: None (can run in parallel with M1.1).

DB migration risk: None.

Existing data: Unaffected. Next populate run will use the correct date.

Rollback plan: Revert one line.

Effort: 30 minutes

Test gate:

Trigger a repopulate from admin Crawl tab.

Inspect the Perplexity call in server logs. Confirm the date matches today.

Confirm no future competition events appear in the past, and no past events appear as "upcoming".

M1.3 — Abort pipeline on Perplexity failure instead of falling back to GPT training data

What changes:

-

ar
tif
ac
ts/
a

pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
In
t
h
e
P
er
pl
ex
ity
c
at
ch
bl
oc
k,
in
st
e
a
d
of
c
o
nti
nu
in
g
wi
th
e
m
pt
y r
es
e
ar
ch
Te
xt,
t
hr
o
w
a
n
er
ro
r t
h
at
c
a

uses a ut o p ul at e At hl et e to e xit e ar ly. S et la st Cr a wl e d At t o re m ai n nu ll. L o g th e fai lur e cl e ar ly wi th a thl et e ID a n d er ro r m es sa g e.

What stays the same: All other pipeline stages. If Perplexity succeeds, behaviour is identical to before.

Dependencies: M1.1 (ensures pipeline uses known-good models before adding error handling logic).

DB migration risk: None.

Existing data: Unaffected. Athletes that were populated with fabricated data are not changed by this milestone — that requires a repopulate, which will use the new pipeline.

Rollback plan: Revert the catch block to its previous form.

Effort: 2 hours

Test gate:

Temporarily point the Perplexity base URL to an invalid endpoint in a test run.

Trigger a populate. Confirm the athlete has lastCrawledAt = null (not a timestamp), intelligenceCount = 0, and no intelligence items are written.

Confirm server logs show the Perplexity failure message and the pipeline abort.

Restore the correct base URL. Confirm a subsequent populate succeeds.

M1.4 — Reject athlete creation if discovery cannot identify sport and nationality

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
In
di
sc
ov
er
At
hl
et
e
Pr
ofi
le,
if
th
e
re
sp
o
ns
e
re
tu
rn
s
sp
or

t:
""
s
p
or
t:
"A
thl
eti
cs
"
(
th
e
fal
lb
ac
k),
o
r
n
ati
o
n
ali
ty:
""

,
d
o
n
ot
si
le
ntl
y
ac
ce
pt
d
ef
a
ult
s.
R
et
ur
n
{
co
nfi
d
e
nc
e:
"I
o
w"
, r
e
q
uir
es
M
a
nu
all
n
p
ut

: t
ru
e
}.
•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
thl
et
es
.ts
: l
n
P
O
S
T
/a
thl
et
es
/di
sc
ov
er
,
if
di
sc
ov
er
y r
et
ur
ns
r
e
q
uir
es
M
a
nu
all
n
p
ut
: t
ru
e,
re
sp
o
n
d
4

2
2
wi
th

a
us
er
-fr
ie
n
dl
y
m
es
sa
g
e:
"C
o
ul
d
n
ot
a
ut
o
m
ati
ca
lly
id
e
nti
fy
[n
a
m
e]
a
s
a
kn
o
w
n
at
hl
et
e.
Pl
e
as
e
pr
ov
id
e
th
eir
s
p
or
t
a
n
d
n
ati

o
n
ali
ty.
"

What stays the same: If discovery succeeds with real data, the flow is identical. The UI error handling already exists for non-200 responses.

Dependencies: M1.1, M1.3.

DB migration risk: None. No athlete stub row is created if discovery fails.

Rollback plan: Remove the confidence check and revert to the original fallback defaults.

Effort: 1.5 hours

Test gate:

Attempt to add an athlete with a very common name and no sport context (e.g., "John Smith"). Confirm a 422 is returned and no athlete row is created.

Attempt to add a well-known athlete (e.g., "Hamish Kerr, High Jump, New Zealand"). Confirm discovery succeeds and pipeline runs normally.

M1.5 — Fix timeline date fallback (stop defaulting to today)

What changes:

- ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
In
t
h
e
ti
m
eli
n
e
ev
e
nt
m
a
p
pi
n
g,
c
h
a
n
g

e
th
e
d
at
e
h
a
n
dli
n
g
fr
o
m:
...

```
date: item.date ? new Date(item.date).toISOString().split("T")[0] : today
...
```

to: if item.date is absent, unparseable, or resolves to an invalid date, skip that timeline event entirely rather than inserting today's date.

What stays the same: Timeline events with valid dates are written exactly as before.

Dependencies: M1.1.

DB migration risk: None. New behaviour only affects new populate runs.

Existing data: Existing timeline events with incorrect dates (set to the populate date) are not automatically fixed — but they will not increase in number from this point forward.

Rollback plan: Revert the date handling logic.

Effort: 1 hour

Test gate:

Trigger a repopulate for an athlete with sparse public data.

Open the Timeline tab. Confirm no events show today's date unless the athlete genuinely has an event today.

Confirm total timeline event count is similar to before (some events may now be skipped — this is correct behaviour, not a regression).

M1.6 — Consolidate duplicate repopulate endpoints

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
thl
et
es
.ts
a
n
d
ar
tif
ac
ts/
a
pi-
se

rv
er
/sr
c/r
o
ut
es
/a
d
mi
n.
ts:
E
xtr
ac
t t
h
e
sh
ar
e
d
re
p
o
p
ul
at
e l
o
gi
c i
nt
o
a
n
e
w
se
rvi
ce
f
u
nc
tio
n
re
p
o
p
ul
at
e
At
hl
et
e(
at
hl
et
el
d:
nu
m
b
er
)
in
a

n
e
w
fil
e
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
re
p
o
p
ul
at
e.t
s.
B
ot
h
ro
ut
es
c
all
t
hi
s f
u
nc
tio
n.
N
o
b
e
h
av
io
ur
c
h
a
n
g
e.

What stays the same: Both API endpoints remain. Their request/response contracts are unchanged. The admin endpoint still has stricter auth. The only change is that the implementation is in one place.

Dependencies: M1.3.

DB migration risk: None.

Rollback plan: Inline the function back into both route files.

Effort: 2 hours

Test gate:

Trigger repopulate from Admin Crawl tab. Confirm it works.

Trigger repopulate from Dossier page (if exposed). Confirm it works.

Confirm `grep -n "DELETE FROM"` in both route files returns zero results (logic has moved to the service).

M1.7 — Standardise social extraction across all code paths

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
C
h
a
n
g
e
so
ci
al
ex
tr
ac
tio
n
fr
o
m
p
er
pl
ex
ity
/s
o
n
ar
'
p
er
pl
ex
ity
/s
o
n
ar
-p
ro
a
n
d
fr
o

m
g
pt
-5
.6
-lu
n
a
!'g
pt
-4
o-
mi
ni.

•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
so
ci
al-
ex
tr
ac
t.t
s (
n
e
w
fil
e)
:
E
xtr
ac
t t
h
e
sh
ar
e
d
P
er
pl
ex
ity

+
G
P
T-
4
o-
mi
ni
so
ci
al
ex

transactional logic onto a single reusable abstraction. All these (auto-populate, add, min, backfill, dosier refresh) call this function.

What stays the same: Route signatures, response shapes, and DB write logic are unchanged.
Dependencies: M1.1, M1.6 (pattern established by shared service extraction).

DB migration risk: None.

Rollback plan: Revert to inline code in each file.

Effort: 2 hours

Test gate:

Run "Backfill Social" from admin. Confirm handles and follower counts are returned.

Click "Refresh from Web" on a dossier. Confirm social data updates.

Add a new athlete. Confirm social data is populated as part of the initial populate.

Confirm perplexity/sonar (non-pro) no longer appears in any social-related code path.

Phase 1 Completion Gate

Before moving to Phase 2, confirm all of the following:

- [] gpt-5.6-lunadonesnot appear anywhere in the codebase
- [] reasoning

ar
a
ny
w
h
er
e
in
th
e
co
d
e
b
as
e
• [
]
R
es
e
ar
ch
p
ro
m
pt
g
e
n
er
at
es
t
o
d
ay
's
d
at
e
dy
n
a
mi
ca
lly
• [
]
P
er
pl
ex
ity
fa
ilu
re
d
o
es
n
ot
cr
e
at
e i
nt
ell
ig

en-
ci-
te-
m-
s-
•[
]
Un-
kn-
o-
wn
at-
hl-
et-
es-
r-
et-
ur-
n-
a-
4-
2-
2-
ra-
th-
er-
t-
h-
a-
n-
d-
ef-
a-
ult-
in-
g-
to-
"
At
hl-
eti-
cs-
"
•[
]
Ti-
m-
eli-
n-
e-
ev-
e-
nt-
s-
n-
o l
o-
n-
g-
er-
d-
ef-
a-
ult-
t-
o

to
d
ay
's
d
at
e
• [
]
S
oc
ial
e
xtr
ac
tio
n
us
es

a
si
n
gl
e
sh
ar
e
d
fu
nc
tio
n
a
n
d
so
n
ar
-p
ro
e
ve
ry
w
h
er
e

Phase 2 — Database Foundations

All schema changes in this phase are additive (new indexes, new nullable columns). Zero risk to existing data. Indexes can be created and dropped without affecting application logic.

M2.1 — Add missing indexes on foreign key columns

What changes:

• li
b/
d
b/
sr
c/
sc
h
e

m
a/i
nt
ell
ig
e
nc
e-i
te
m
s.t
s:
A
d
d i
n
d
ex
o
n
at
hl
et
el
d,
a
d
d i
n
d
ex
o
n
di
sc
ov
er
e
d
At
•li
b/
d
b/
sr
c/
sc
h
e
m
a/
ti
m
eli
n
e-
ev
e
nt
s.t
s:
A
d
d i
n
d
ex
o
n

at
hl
et
el
d
•li
b/
d
b/
sr
c/
sc
h
e
m
a/
co
nt
ac
ts.
ts:
A
d
d i
n
d
ex
o
n
at
hl
et
el
d
•li
b/
d
b/
sr
c/
sc
h
e
m
a/
co
m
p
eti
tio
ns
.ts
:
A
d
d i
n
d
ex
o
n
at
hl
et
el
d,
a
d
d i

n
d
ex
o
n
d
at
e
•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

What stays the same: All queries, all routes, all data. Indexes are invisible to application code.

Dependencies: Phase 1 complete.

DB migration risk: Low. Creating indexes on existing tables locks them briefly (milliseconds on a small table). For a table with fewer than 10,000 rows, this is imperceptible. No data is changed.

Existing data: Fully preserved.

Rollback plan: Drop the indexes. DROP INDEX CONCURRENTLY idx_name; — zero data impact.

Effort: 1 hour

Test gate:

Open a dossier page. Confirm all tabs still load correctly.

Check EXPLAIN ANALYZE on SELECT * FROM intelligence_items WHERE athlete_id = \$1 — confirm it shows Index Scan not Seq Scan.

Confirm pnpm --filter @workspace/db run db:push reports no destructive changes.

M2.2 — Add unique constraint on athlete name (case-insensitive)

What changes:

• li
b/
d
b/
sr
c/
sc
h
e
m
a/
at
hl
et

es
.ts
:
A
d
d
u
ni
q
u
el
n
d
ex
(i
dx

—
at
hl
et
es

—
n
a
m
e

—
u
ni
q
u
e')
.o
n(
sq
llo
w
er
(n
a
m
e)
)

•

R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p

us
h

What stays the same: All existing data. The unique index is created only if no duplicates exist. If duplicates are found, they must be resolved manually first (check with: `SELECT lower(name), COUNT() FROM athletes GROUP BY lower(name) HAVING COUNT() > 1`).

Dependencies: M2.1.

DB migration risk: Low–Medium. If duplicate athlete names exist in the database, the migration will fail. Pre-check must be run before applying. If duplicates exist, the admin must merge or delete them first via the admin panel.

Existing data: No data is changed. The constraint only prevents future duplicates.

Rollback plan: `DROP INDEX idx_athletes_name_unique;` — zero data impact.

Effort: 1 hour (plus time to resolve any pre-existing duplicates)

If this returns rows, resolve them before applying the index.

Pre-check SQL:

Test gate:

Attempt to add an athlete that already exists (same name, different capitalisation). Confirm the system returns the existing athlete rather than creating a duplicate.
Confirm the pre-check SQL returns zero rows after the migration.

M2.3 — Add `avatar\_url\_locked` column to athletes

What changes:

- li
b/
d
b/
sr
c/
sc
h
e
m
a/
at
hl
et
es
.ts
:
A
d
d
av
at
ar
Ur
IL
oc
ke
d
b
o
ol
e
a
n
d
ef
a
ult
fa
ls
e
•
ar
tif
ac
ts/

a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
d
mi
n.
ts:
I
n
th
e
p
h
ot
o
b
ac
kfi
ll r
o
ut
e,
a
d
d
W
H
E
R
E
av
at
ar
-
ur
l_l
oc
ke
d
=
fal
se
t
o
th
e
at
hl
et
e
se
le
cti
o
n
q
u
er
y
•
R

un
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

What stays the same: All existing photos. No existing avatarUrl values are changed. avatarUrlLocked defaults to false, so existing behaviour is unchanged.

Dependencies: M2.1.

DB migration risk: None. Adding a nullable/defaulted column to an existing table is the safest possible schema change in PostgreSQL.

Existing data: All existing avatarUrl values preserved exactly. All athletes default to avatarUrlLocked = false.

Rollback plan: ALTER TABLE athletes DROP COLUMN avatar_url_locked; — zero data impact.

Effort: 1 hour

Test gate:

Manually set avatar_url_locked = true for one athlete directly in the DB.

Run "Backfill Photos" from admin. Confirm the locked athlete's photo is not changed.

Confirm unlocked athletes are still processed by the backfill.

Phase 2 Completion Gate

• [
]
All
F
K
co
lu
m
ns
h
av
e i
n
d
ex
es
(v
er
ify
w
ith
\
d
ta
bl

e
n
a
m
e
in
ps
ql)
•[
]
E
X
P
L
AI
N
A
N
A
LY
Z
E
o
n
a
ny

W
H
E
R
E
at
hl
et
e
_i
d
=
\$
1
q
u
er
y
sh
o
w
s i
n
d
ex
s
ca
n
•[
]
D
u
pli
ca
te
a
thl
et
e
n
a
m

es
p
ro
d
uc
e
a
d
e
d
u
p
re
su
lt,
n
ot

a
n
e
w
ro
w
• [
]
P
h
ot
o
b
ac
kfi
ll
sk
ip
s l
oc
ke
d
at
hl
et
es

Phase 3 — Source Registry

Creates the sources table — a new table with no connections to existing tables yet. Purely additive. The existing system is completely unaffected.

M3.1 — Create `sources` registry table and seed it

What changes:

- N
e
w
fil
e:
lib
/d
b/
sr
c/
sc
h

e
m
a/
so
ur
ce
s.t
s
—
d
efi
n
es
t
h
e
so
ur
ce
s t
a
bl
e
(d
o
m
ai
n,
di
sp
la
y_
n
a
m
e,
tie
r,
so
ur
ce
_t
yp
e,
sp
or
t_
sc
o
p
e,
is
—
of
fic
ial
,
n
ot
es
, c
re
at
e
d
—
at
, l
as

t_
ve
rifi
e
d)
•
N
e
w
fil
e:
lib
/d
b/
sr
c/
se
e
ds
/s
o
ur
ce
s.t
s
—
in
se
rt
s t
h
e i
nit
ial
3
0
—
4
0
kn
o
w
n
so
ur
ce
s (
W
or
ld
At
hl
eti
cs
,
U
Cl
,
W
iki
p
e
di
a,
e
tc.
)
wi
th

t
h
eir
ti
er
a
ss
ig
n
m
e
nt
s
• li
b/
d
b/
sr
c/
sc
h
e
m
a/i
n
d
ex
.ts
:
E
xp
or
t t
h
e
n
e
w
ta
bl
e
•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

•
Run
the
seed
script

What stays the same: Every existing table and route. Nothing reads from sources yet.

Dependencies: Phase 2 complete.

DB migration risk: None. New table, no connections.

Existing data: Unaffected.

Rollback plan: DROP TABLE sources; — zero impact on existing system.

Effort: 1 day (schema: 1 hour; seed data research and entry: 6 hours)

Initial sources to seed (sample):

Domain	Display name	Tier	Type
worldathletics.org	World Athletics	1	api
uci.org	UCI	1	api
fis-ski.com	FIS	1	web
aquatics.sport	World Aquatics	1	api
olympics.com	International Olympic Committee	1	web
athleticsnz.org.nz	Athletics New Zealand	1	web
cyclingnewzealand.nz	Cycling New Zealand	1	web
firstcycling.com	First Cycling	4	web
procyclingstats.com	ProCyclingStats	4	web
tilastopaja.eu	Tilastopaja	4	web
swimrankings.net	SwimRankings	4	web
bbc.com/sport	BBC Sport	6	web
reuters.com	Reuters	6	web
wikipedia.org	Wikipedia	7	web
perplexity.ai	Perplexity (AI estimate)	7	ai_estimated
twitter.com	X (Twitter)	7	social

instagram.com

Instagram

social

tiktok.com	TikTok	7	social
youtube.com	YouTube	7	social

Test gate:
SELECT COUNT(*) FROM sources; — confirm seed data is present
SELECT * FROM sources WHERE tier = 1; — confirm all Tier 1 sources are present
Confirm the application still loads and all dossiers still display correctly

M3.2 — Add nullable `source\_id` FK to `intelligence\_items`

What changes:

• li
b/
d
b/
sr
c/
sc
h
e
m
a/i
nt
ell
ig
e
nc
e-i
te
m
s.t
s:
A
d
d
so
ur
ce
ld
u
ui
d
re
fe
re
nc
es
s
o
ur
ce
s(i
d)

—
n
ull
a
bl
e,
n
o

d
ef
a
ult
•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
W
h
e
n
wr
iti
n
g i
nt
ell
ig
e
nc
e i
te
m
s,
at
te
m
pt
t
o
m
at
ch
s
o
ur
ce
D
o
m
ai
n
a
g
ai
ns
t t
h
e

so
ur
ce
s t
a
bl
e.
If
a
m
at
ch

is
fo
u
n
d,
p
o
p
ul
at
e
so
ur
ce
ld
.

If
n
o
m
at
ch
, l
e
av
e
nu
ll.
T
h
e
pi
p
eli
n
e
d
o
es
n
ot
fa
il
if
so
ur
ce

is
n
ot
fo
u
n
d.
•

R
u
n
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

What stays the same: The existing sourceDomain text column remains. The new sourceId column is supplementary and nullable. All existing queries continue to work.

Dependencies: M3.1.

DB migration risk: None. Adding a nullable FK column. Existing rows get sourceId = null. The FK constraint only applies to new writes.

Existing data: All existing intelligence items retain their sourceDomain text value. sourceId defaults to null for all existing rows.

Rollback plan: ALTER TABLE intelligence_items DROP COLUMN source_id;

Effort: 2 hours

Test gate:

Add a new athlete. Confirm intelligence items are created.

SELECT source_id, source_domain FROM intelligence_items WHERE source_id IS NOT NULL LIMIT 5; — confirm some items now have source_id populated

Confirm the dossier Intelligence tab still loads and displays all items correctly

M3.3 — Add nullable `source\_id` FK to `timeline\_events`, `contacts`, `competitions`

What changes:

•
S
a
m
e
p
at
te
rn
a
s
M
3.
2
a
p
pli
e
d
to
t

h
e
re
m
ai
ni
n
g
th
re
e
ta
bl
es

.
•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: M3.2.

DB migration risk: None. Identical pattern to M3.2.

Existing data: Preserved. All existing rows get sourceId = null.

Rollback plan: Drop the three columns individually.

Effort: 2 hours

Test gate:

•
S
a
m
e
p
at
te
rn
a
s
M
3.
2
fo
r
e
ac

h
ta
bl
e
•
C
o
nfi
r
m
Ti
m
eli
n
e,
C
o
nt
ac
ts,
a
n
d
C
o
m
p
eti
tio
ns
t
a
bs
al
ll
o
a
d
co
rr
ec
tly

Phase 4 — Population Status Tracking

Adds real pipeline status visibility. This directly unblocks the frontend polling problem identified in the audit (M6 from audit). Small, safe, high-value.

M4.1 — Add `population\_status` column to athletes

What changes:

• li
b/
d
b/
sr
c/
sc
h
e
m
a/
at
hl
et
es

.ts
:
A
d
d
p
o
p
u
l
a
t
i
o
n
S
t
a
t
u
s
t
e
x
t
d
e
f
a
u
l
t
'i
d
l
e'

- V
a
l
i
d
v
a
l
u
e
s
:
'i
d
l
e'
|
'
q
u
e
u
e
d'
|
'r
e
s
e
a
r
c
h
i
n
g'
|
'
e
x
t
r
a
c
t
i
n
g'
|
'
c
o
m
p
l
e
t
e'
|
'f
a
i
l
e'
d'

- R
u
n

p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: Phase 3 complete.

DB migration risk: None. Additive column with default.

Existing data: All existing athletes get populationStatus = 'idle'.

Rollback plan: ALTER TABLE athletes DROP COLUMN population_status;

Effort: 30 minutes

M4.2 — Wire pipeline to update `population\_status` at each stage

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
re
p
o
p
ul
at
e.t
s (
fr
o
m
M
1.
6)
:
A
d
d
st
at

us
u
p
d
at
es
a
t
e
ac
h
pi
p
eli
n
e
st
a
g
e:

- On job start: set population on Status

= 'queueued'

- On Perplexity call start: set population

n
St
at
us

= '
re
se
ar
ch
in
g'

•
O
n
G
P
T
ex
tr
ac
tio
n
st
ar
t:
se
t
p
o
p
ul
ati
o
n
St
at
us

= '
ex
tr
ac
tin
g'

•
O
n
su
cc
es
s:
se
t
p
o
p
ul
ati
o
n
St
at
us

= '
co
m
pl

et
e'
•
O
n
fai
lur
e:
se
t
p
o
p
ul
ati
o
n
St
at
us

=
fai
le
d'

Dependencies: M4.1, M1.6.

DB migration risk: None.

Effort: 2 hours

M4.3 — Update frontend polling to use `population\_status`

What changes:

•
ar
tif
ac
ts/
w
e
b/
sr
c/
p
a
g
es
/D
os
si
er
P
a
g
e.t
sx
:
C
h
a
n
g
e
th
e
p
oll
in

g
te
r
mi
n
ati
o
n
co
n
dit
io
n
fr
o
m
ti
m
e-
b
as
e
d
(6
0s
ti
m
e
o
ut
) t
o
st
at
us
-b
as
e
d:
te
r
mi
n
at
e
w
h
e
n
p
o
p
ul
ati
o
n
St
at
us
=
=
='
co
m
pl
et
e'
or
p

o
p
ul
ati
o
n
St
at
us
=
=
=
'
fai
le
d'.
S
h
o
w
a
pr
o
gr
es
s I
a
b
el
b
as
e
d
o
n
th
e
cu
rr
e
nt
st
at
us
v
al
u
e.
•
ar
tif
ac
ts/
w
e
b/
sr
c/
p
a
g
es
/A
d
mi
n
P
a
g
e.t

sx
:
S
h
o
w
p
o
p
u
l
a
t
i
o
n
St
at
us

in
th
e
Cr
a
wl
t
a
b
at
hl
et
e
ta
bl
e.

Dependencies: M4.2.

DB migration risk: None.

Effort: 2 hours

Phase 4 test gate:

Add a new athlete. Watch the dossier page. Confirm the status label progresses through "Queued !" Researching !" Extracting !" Complete" rather than just spinning for 60s.
Temporarily break the API URL. Add a new athlete. Confirm the dossier shows "Failed" rather than "No data found" after a timeout.
In admin Crawl tab, confirm each athlete shows their population status.

Phase 5 — Evidence Foundation

This is the first significant architectural milestone. New tables are created and the pipeline begins writing to them in parallel with existing tables. Existing tables are completely untouched. This is purely additive.

M5.1 — Create `evidence\_items` table

What changes:

- N
e
w
fil
e:
lib
/d
b/
sr
c/
sc
h

e
m
a/
ev
id
e
nc
e-i
te
m
s.t
s
—
f
ull
s
ch
e
m
a
as
d
efi
n
e
d
in
th
e
Te
ch
ni
ca
l
D
es
ig
n
d
oc
u
m
e
nt
• li
b/
d
b/
sr
c/
sc
h
e
m
a/i
n
d
ex
.ts
:
E
xp
or
t it
•
R
u
n
p

n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: Phase 4 complete. M3.1 (requires sources table).

DB migration risk: None. New table.

Existing data: Unaffected.

Rollback plan: DROP TABLE evidence_items;

Effort: 3 hours (schema definition + indexes)

M5.2 — Create `evidence\_supporting\_sources` table

What changes:

-

N
e
w
fil
e:
lib
/d
b/
sr
c/
sc
h
e
m
a/
ev
id
e
nc
e-
su
p
p
or
tin
g-
so
ur
ce
s.t
s

-

E

xp
or
t it

- R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: M5.1.

DB migration risk: None.

Effort: 30 minutes

M5.3 — Dual-write: pipeline writes to `evidence\_items` alongside existing tables

What changes:

- ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s(
a
n
d
re
p
o

p
ul
at
e.t
s):
A
ft
er
e
ac
h
D
B
in
se
rt
to
in
tel
lig
e
nc
e
_it
e
m
s,
ti
m
eli
n
e
—
ev
e
nt
s,
co
nt
ac
ts,
c
o
m
p
eti
tio
ns
,
al
so
in
se
rt
a
co
rr
es
p
o
n
di
n
g
ro
w
int
o
ev

id
e
nc
e
_it
e
m
s.
T
h
e
tw
o
wr
ite
s
h
a
p
p
e
n
in
th
e
sa
m
e
D
B
tr
a
ns
ac
tio
n
w
h
er
e
p
os
si
bl
e.
•
T
h
e
ex
ist
in
g
ta
bl
es
c
o
nti
nu
e
to
r
ec
ei
ve
w
rit
es

a
s
b
ef
or
e.
N
ot
hi
n
g
is
re
m
ov
e
d.

•
A
fai
le
d
ev
id
e
nc
e
_it
e
m
s
wr
ite
lo
gs

a
w
ar
ni
n
g
bu
t
d
o
es
N
O
T
rol
l b
ac
k t
h
e
ex
ist
in
g
ta
bl
e
wr
ite
. T
h
e
ex

ist
in
g
sy
st
e
m
m
us
t
n
ev
er
b
e
d
e
gr
a
d
e
d
by
t
h
e
n
e
w
sy
st
e
m.

Key mapping:

Existing table	evidence_items fact_type
intelligence_items (category: results_rankings)	competition_result
intelligence_items (category: media_interviews)	media_appearance
intelligence_items (category: sponsorships)	sponsorship
intelligence_items (category: career_changes)	career_transition
timeline_events	timeline_event
contacts (category: coaching)	coach
contacts (category: management)	agent
contacts (category: sponsorship)	sponsor_contact
competitions	competition_result (with result_value)

Dependencies: M5.2, M3.2.

DB migration risk: Low. New writes added. Existing writes unchanged. If the evidence write fails, a warning is logged.

Existing data: Unaffected.

Rollback plan: Remove the dual-write code from the pipeline. Evidence items written so far are left in place (harmless) or truncated with TRUNCATE evidence_items;.

Effort: 3 days

Test gate:

Add a new athlete. Confirm:

- i
nt
ell
ig
e
nc
e
_it
e
m
s
is
p
o
p
ul
at
e
d
(a
s
b
ef
or
e)
- ev
id
e
nc
e
_it
e
m
s
is
al
so
p
o
p
ul
at
e
d
wi
th
c
or
re
sp
o
n
di
n
g
ro
w
s
- T
h
e

d
os
si
er
p
a
g
e
di
sp
la
ys
c
or
re
ctl
y (
re
a
ds
fr
o
m
ex
ist
in
g
ta
bl
es
)

SELECT COUNT(*) FROM evidence_items WHERE athlete_identity_id IS NULL; — confirm all evidence items have athlete linkage
Confirm zero errors in server logs related to evidence writes

M5.4 — Backfill existing `intelligence\_items` !' `evidence\_items`

What changes:

- N
e
w
o
n
e-
of
fs
cri
pt
: a
rti
fa
ct
s/
a
pi-
se
rv
er
/s
cri
pt
s/
b
ac
kfi
ll-
ev

id
e
nc
e.t
s
•
S
cri
pt
r
e
a
ds
al
le
xi
sti
n
g i
nt
ell
ig
e
nc
e
_it
e
m
s,
ti
m
eli
n
e
_ev
e
nt
s,
co
nt
ac
ts,
c
o
m
p
eti
tio
ns
a
n
d
cr
e
at
es
c
or
re
sp
o
n
di
n
g
ev
id
e

nc
e
_it
e
m
s r
o
w
s.
U
se
s
P
er
pl
ex
ity
A
l
es
ti
m
at
e
as
t
h
e
so
ur
ce
(
Ti
er
7
)s
in
ce
n
o
b
et
te
r
pr
ov
e
n
a
nc
e
ex
ist
s f
or
le
g
ac
y
d
at
a.
•
S
cri
pt

is
id

e
m
p
ot
e
nt
:
if r
u
n
tw
ic
e,
it
d
o
es
n
ot
cr
e
at
e
d
u
pli
ca
te
s (
ch
ec
ks
fo
r
ex
ist
in
g
ev
id
e
nc
e i
te
m
wi
th
m
at
ch
in
g
at
hl
et
e
+
fa
ct
_t
yp
e
+
fa
ct
_v
al
u
e

+
fa
ct

—
d
at
e
b
ef
or
e i
ns
er
tin
g)
:
•
R
u
n
o
nc
e
fr
o
m
a
d
mi
n
sh
ell
a
ft
er
v
er
ify
in
g
M
5.
3
is
st
a
bl
e.

Note on legacy data quality: The backfilled evidence items will have confidence set to 35 (AI estimate) and verification_status = 'unverified'. This is honest — legacy data came from an AI pipeline with no source verification. Future repopulates will create higher-confidence evidence items for the same athletes.

Dependencies: M5.3 running stably for at least 3 new athlete populates.

DB migration risk: Low. The script only inserts into the new evidence_items table. No existing data is changed. If the script fails mid-run, simply re-run it (idempotent).

Existing data: Preserved exactly. The backfill reads from existing tables, does not modify them.

Rollback plan: TRUNCATE evidence_items; — removes all evidence items. The existing system continues to work from its own tables. Only the new evidence layer is affected.

Effort: 1 day

Test gate:

SELECT COUNT() FROM evidence_items; should be roughly equal to SELECT COUNT() FROM intelligence_items + SELECT COUNT() FROM timeline_events + SELECT COUNT() FROM contacts + SELECT COUNT(*) FROM competitions

Run the backfill script a second time. Confirm the count does not increase (idempotency check).

All dossier pages continue to work normally.

Phase 5 Completion Gate

•[
]
All
n
e
w
at
hl
et
es
h
av
e
ev
id
e
nc
e
_it
e
m
s
p
o
p
ul
at
e
d
by
t
h
e
pi
p
eli
n
e
•[
]
All
e
xi
sti
n
g
at
hl
et
es
h
av
e
ev
id
e
nc
e
_it
e
m
s
b
ac
kfi
lle
d

•[
]
N
o
ex
ist
in
g
fu
nc
tio
n
ali
ty
is
br
ok
e
n
•[
]
E
vi
d
e
nc
e i
te
m
s
h
av
e
co
rr
ec
t s
o
ur
ce
li
nk
a
g
e
w
h
er
e
d
o
m
ai
n
m
at
ch
es
t
h
e
so
ur
ce
s r
e
gi
str
y

- []
D
u
al-
wr
ite
fa
ilu
re
d
o
es
n
ot
b
re
ak
t
h
e
m
ai
n
p
o
p
ul
at
e
pi
p
eli
n
e

Phase 6 — Identity Engine

Creates permanent athlete identities. Initially runs alongside the existing athletes table. Neither table is removed at this stage.

M6.1 — Create `athlete\_identities` table

What changes:

- N
e
w
fil
e:
lib
/d
b/
sr
c/
sc
h
e
m
a/
at
hl
et
e-i
d
e

nti
tie
s.t
s
—
f
ull
s
ch
e
m
a
as

in
Te
ch
ni
ca
l
D
es
ig
n

•
E
xp
or
t it

•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d:
b:
p
us
h

•
A
d
d
at
hl
et
e
_i
d
e
nti

ty
_i
d
u
ui
d
re
fe
re
nc
es
a
thl
et
e
_i
d
e
nti
tie
s(i
d)
n
ull
a
bl
e
co
lu
m
n
to
a
thl
et
es
t
a
bl
e
(t
h
e
br
id
g
e
co
lu
m
n
th
at
li
nk
s
ol
d
a
n
d
n
e
w
sy
st
e
m
s)

Dependencies: Phase 5 complete.

DB migration risk: None. New table plus nullable FK column on athletes.

Existing data: All existing athletes rows get athleteIdentityId = null. This is correct — they have not been linked to an identity record yet (that happens in M6.3).

Rollback plan: Drop athlete_identities table and athlete_identity_id column from athletes.

Effort: 3 hours

M6.2 — New athletes create an identity record (dual-write)

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
thl
et
es
.ts
(
P
O
S
T
/a
thl
et
es
/di
sc
ov
er
):
Af
te
r c
o
nfi
r
mi
n
g
th
e
at
hl
et
e
is
n
e
w,
cr
e
at
e

a
n
at
hl
et
e
_i
d
e
nti
tie
s r
ec
or
d
b
ef
or
e i
ns
er
tin
g
th
e
at
hl
et
es
r
o
w.
S
et
a
thl
et
es
.a
thl
et
el
d
e
nti
tyl
d
to
t
h
e
n
e
w
id
e
nti
ty
U
UI
D.
•
T
h
e
fin
g
er
pr

int
u
ni
q
u
e
n
es
s
ch
ec
k r
u
ns
a
g
ai
ns
t
at
hl
et
e
_i
d
e
nti
tie
s f
irs
t.
If
th
e
fin
g
er
pr
int
al
re
a
dy
e
xi
st
s,
th
e i
d
e
nti
ty'
s
id
is
re
us
e
d.

What stays the same: The athletes table continues to receive all writes. The athlete_identities record is additional, not a replacement.

Dependencies: M6.1.

DB migration risk: Low. One additional insert per new athlete.

Rollback plan: Remove the identity creation code. Future athletes will have athleteidentityId = null.

Effort: 1 day

Test gate:

Add a new athlete. Confirm a corresponding row exists in athlete_identities.
Attempt to add the same athlete again (same name/sport/nationality). Confirm the existing identity is returned and no duplicate is created in either table.
All existing athletes continue to work normally (their athleteIdentityId is null — that is expected at this stage).

M6.3 — Backfill existing athletes with identity records

What changes:

- New script : a rti facts/ a pi-se rv er /s cri pt s/ b ac kfi ll-i d e nti tie s.t s
- For each existing ing at hl et es r o w w h er e at hl et el d e nti tyl

d
IS
N
U
L
L:

- Create an athlete identifier record from the athlete's name, sport, nationality
- Set athletes .athlete_id = new

i
d
e
n
t
i
t
y
i
d

- S
c
r
i
p
t

is
id
e
m
p
o
t
e
n
t

.

- Af
te
r
r
u
n
n
i
n
g,
u
p
d
at
e
ev
id
e
nc
e
_it
e
m
s.
at
hl
et
e
_i
d
e
n
t
i
t
y
_i
d
to
p
oi
nt
t
o
th
e
n
e
w
id
e
n
t
i

ty
U
UI
D
s (
cu
rr
e
ntl
y
nu
ll f
or
b
ac
kfi
lle
di
te
m
s).

Dependencies: M6.2 running stably, M5.4 complete.

UPDATE athletes SET athlete_identity_id = null;

Dependencies: M6.2 running stably, M5.4 complete.
Migration risk: Medium. This script updates athletes.athlete_identity_id for existing rows. If interrupted, some rows are updated and others are not — but the script is idempotent, so re-running it is safe.

Zero data loss. The system falls back to operating with null identity IDs, which is the state before this script ran.

Rollback plan:

Effort: 1 day

Test gate:

```
SELECT COUNT(*) FROM athletes WHERE athlete_identity_id IS NULL; — should return 0 after the script
SELECT COUNT(*) FROM evidence_items WHERE athlete_identity_id IS NULL; — should return 0 after the script
```

Confirm athlete identity fingerprint uniqueness: `SELECT identity_fingerprint, COUNT() FROM athlete_identities GROUP BY identity_fingerprint HAVING COUNT() > 1;` — should return zero rows

M6.4 — Add identity deduplication to the discovery flow

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
thl
et
es
.ts
:
B
ef
or
e
th
e
cu
rr
e
nt

d
u
pli
ca
te
c
h
ec
k(
S
E
L
E
C
T
*
F
R
O
M
at
hl
et
es

W
H
E
R
E
n
a
m
e
=
\$
1)
,
a
d
d
a
fin
g
er
pr
int
c
h
ec
k
a
g
ai
ns
t
at
hl
et
e
_i
d
e
nti
tie
s.
S
h
o

w
a
UI
w
ar
ni
n
g
if
a
n
a
m
e-
si
mi
lar
id
e
nti
ty
alr
e
a
dy
e
xi
st
s.

•

A
d
d
a
bir
th
y
e
ar
fi
el
d
to
t
h
e
n
e
w
at
hl
et
e
cr
e
ati
o
n
fo
r
m
(o
pti
o
n
al,
fo
r
di

sa
m
bi
g
u
ati
o
n)
.

Dependencies: M6.3.

DB migration risk: None.

Effort: 1 day

Phase 6 test gate:

• [
]
E
ve
ry
a
thl
et
e
ro
w
h
as

a
co
rr
es
p
o
n
di
n
g i
d
e
nti
ty
re
co
rd

• [
]
E
ve
ry
e
vi
d
e
nc
e i
te
m
h
as
a
n
at
hl
et
e
_i
d

e
nti
ty
_i
d
•[
]
Cr
e
ati
n
g
a
d
u
pli
ca
te
a
thl
et
e
(s
a
m
e
fin
g
er
pr
int
) r
et
ur
ns
e
xi
sti
n
g i
d
e
nti
ty
•[
]
D
os
si
er
p
a
g
es
,
a
d
mi
n
p
a
n
el,
a
n
d
all
e
xi

stingatures work normally

Phase 7 — Social Engine

Replaces flat social columns with a proper handle registry and snapshot history. The existing columns remain in place and continue to be used by the frontend until Phase 11.

M7.1 — Create `social\_handles` and `social\_snapshots` tables

What changes:

- New files added: lib/b/d/b/sr/c/sc/health_metrics, lib/b/d/b/sr/c/sc/health_metrics.ts, lib/b/d/b/sr/c/sc/health_metrics.d.ts, lib/b/d/b/sr/c/sc/health_metrics.js

ps
h
ot
s.t
s
•
E
xp
or
t t
h
e
m
•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: Phase 6 complete.
DB migration risk: None. New tables.
Effort: 2 hours

M7.2 — Backfill existing social data into new tables

What changes:

•
N
e
w
sc
ri
pt
: a
rti
fa
ct
s/
a
pi-
se
rv
er
/s
cri

pts/
b
ac
kfi
ll-
so
ci
al.
ts

•
F
or
e
ac
h
at
hl
et
e
wi
th
n
o
n-
nu
ll
so
ci
al
h
a
n
dl
es
/fo
llo
w
er
s,
cr
e
at
e:

•
so
ci
al

—
h
a
n
dl
es
r
o
w
s f
or
e
ac
h
kn
o
w
n
pl
at
fo

r
m
•
O
n
e
s
o
c
i
a
l
_s
n
a
p
s
h
o
t
s
r
o
w
p
e
r
p
l
a
t
f
o
r
m
u
s
i
n
g
t
h
e
c
u
r
r
e
n
t
v
a
l
u
e
s
(
a

a
h
i
s
t
o
r
i
c
a
l
b
a
s
e
l
i
n
e)

•
S
e
t
v
e
r
i
f
i
c
a
t
i
o
n
_s
t
a
t
u
s
'
u
n
v
e
r

ifi
e
d'
fo
r
all
b
ac
kfi
lle
d
h
a
n
dl
es
(t
h
ey
w
er
e
Al
-di
sc
ov
er
e
d,
n
ot
v
er
ifi
e
d)

•
S
et
d
at
a
_s
o
ur
ce

= '
le
g
ac
y_
b
ac
kfi
ll'
fo
r
all
b
ac
kfi
lle
d
sn
a
ps
h
ot

s
•
S
cri
pt

is
id
e
m
p
ot
e
nt

Dependencies: M7.1, M6.3 (identity records needed for FK).

DB migration risk: Low. New tables only. Existing athletes columns unchanged.

Rollback plan: TRUNCATE social_handles, social_snapshots;

Effort: 1 day

M7.3 — New social collection writes to `social\_snapshots`

What changes:

•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
so
ci
al-
ex
tr
ac
t.t
s (
fr
o
m
M
1.
7)
:
Af
te
r
wr
iti
n
g
so
ci
al
d
at
a
to
t
h
e

exist-
ing
g
at
hl
et
es
c
ol
u
m
ns
(
as
b
ef
or
e)
,
al
so
w
rit
e
to
s
oc
ial
-
h
a
n
dl
es
a
n
d
so
ci
al
_s
n
a
ps
h
ot
s.
C
o
m
p
ut
e
7-
d
ay
a
n
d
3
0-
d
ay
d
elt
as
fr
o

m
th
e
m
os
t r
ec
e
nt
p
re
vi
o
us
s
n
a
ps
h
ot
.

Dependencies: M7.2.

DB migration risk: None. Additional writes.

Effort: 1.5 days

M7.4 — Frontend: show verified vs estimated labels on social data

What changes:

- ar
tif
ac
ts/
w
e
b/
sr
c/
p
a
g
es
/D
os
si
er
P
a
g
e.t
sx
(
S
oc
ial
M
e
di
a
ta
b)
:
A
d
d
a
n

e
w
A
Pl
c
all
t
o
G
E
T
/a
pi/
at
hl
et
es
/:i
d/
so
ci
al-
h
a
n
dl
es
(
n
e
w
e
n
d
p
oi
nt
).
S
h
o
w
th
e
a
p
pr
o
pr
iat
e l
a
b
el
(' v
er
ifi
e
d
/ "H
es
ti
m
at
e
d
/ &
u
nv

er
ifi
e
d)
al
o
n
gs
id
e
e
ac
h
fol
lo
w
er
c
o
u
nt
.
S
h
o
w
th
e
sn
a
ps
h
ot
d
at
e
(")
as
o
f J
u
n
2
0
2
5"
).
•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/a
thl
et
es
.ts
:
A

d
d
G
E
T
/a
thl
et
es
/:i
d/
so
ci
al-
h
a
n
dl
es
e
n
d
p
oi
nt
t
h
at
r
et
ur
ns
s
oc
ial
h
a
n
dl
es
a
n
d
th
eir
m
os
t r
ec
e
nt
s
n
a
ps
h
ot
.

What stays the same: The existing flat columns (instagramFollowers etc.) continue to power this UI as the fallback. The new endpoint data overlays on top. If the new endpoint returns no data, the existing display is shown unchanged.

Dependencies: M7.3.

DB migration risk: None. New read endpoint only.

Effort: 1.5 days

Test gate for M7.4:

Confirmed athlete has social handles. Open Dossier !' Social Media tab. Confirm follower counts show with appropriate verified/estimated labels.

Confirm "as of [date]" timestamp is shown.
Confirm the tab still works correctly if social_handles has no data (falls back to existing display).

M7.5 — Wire up Twitter API for verified follower counts

What changes:

•
S
et
T
W
IT
T
E
R

-
B
E
A
R
E
R

-
T
O
K
E
N
e
nv
iro
n
m
e
nt
s
ec
re
t(
m
us
t
b
e
o
bt
ai
n
e
d
—
a
u
dit
fi
n
di
n
g
H
2
n
ot
e
d
it
is
n

ot
c
ur
re
ntl
y
se
t)

•

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
so
ci
al-
ex
tr
ac
t.t
s:
F
or
a
thl
et
es
w
ith

a
T
wi
tt
er
h
a
n
dl
e,
ca
ll t
h
e
X
A
PI
v
2
dir
ec
tly
t
o
g
et
t
h
e
a
ut

h
or
ita
tiv
e
fol
lo
w
er
c
o
u
nt
. St
or
e
as
d
at
a
_s
o
ur
ce

= '
tw
itt
er
—
a
pi'
, i
s_
ve
rifi
e
d
=
tr
u
e,
co
nfi
d
e
nc
e
=
9
5.

Note: This milestone is conditional on TWITTER_BEARER_TOKEN being available. If not available, skip and mark M7.5 as blocked.

Dependencies: M7.4.

DB migration risk: None.

Effort: 1 day (if token available)

Phase 7 test gate:

• [
]
All
e
xi
sti
n
g
at

hl
et
es
h
av
e
so
ci
al
_h
a
n
dl
es
r
o
w
s
•[
]
At
le
as
t
o
n
e
so
ci
al
_s
n
a
ps
h
ot
s r
o
w
ex
ist
s
p
er
a
thl
et
e
p
er
pl
at
fo
r
m
•[
]
S
oc
ial
M
e
di
a
ta
b
sh
o

w
s
ve
rifi
ca
tio
n l
a
b
el
s
• [
]
R
u
n
n
ni
n
g
"B
ac
kfi
ll
S
oc
ial
"
cr
e
at
es
n
e
w
so
ci
al
_s
n
a
ps
h
ot
s r
o
w
s (
n
ot
ju
st
u
p
d
at
es
e
xi
sti
n
g
at
hl
et
es
c
ol
u
m

ns
)
• [
]
T
h
e
at
hl
et
es
t
a
bl
e
so
ci
al
co
lu
m
ns
st
ill
h
av
e
co
rr
ec
t v
al
u
es
(
d
u
al-
wr
ite
c
o
nfi
r
m
e
d)

Phase 8 — Job Queue

Replaces the detached background promise with a durable, recoverable job queue backed by PostgreSQL. No external infrastructure required.

M8.1 — Create `crawl\_jobs` table

What changes:

- N
e
w
fil
e:
lib
/d
b/
sr

c/
sc
h
e
m
a/
cr
a
wl
-jo
bs
.ts

—
f
ull
s
ch
e
m
a
as

in
Te
ch
ni
ca
l
D
es
ig
n

•
E
xp
or
t it

•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or
ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

Dependencies: Phase 7 complete.

DB migration risk: None.

Effort: 2 hours

M8.2 — Wrap `autoPopulateAthlete`` in the job queue

What changes:

- ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
re
p
o
p
ul
at
e.t
s:
In
st
e
a
d
of
di
re
ctl
y
ca
lli
n
g
a
ut
o
P
o
p
ul
at
e
At
hl
et
e,
in
se
rt
a
cr
a
wl
_j
o
bs
r
o
w
wi
th

jo
b
_t
yp
e'
=
ful
l_
p
o
p
ul
at
e',
st
at
us

= '
q
u
e
u
e
d'.

•
N
e
w
fil
e:
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
jo
b-
w
or
ke
r.t
s:
A
w
or
ke
r t
h
at
p
oll
s
cr
a
wl
_j
o
bs

W
H

E
R
E
st
at
us

= '
q
u
e
u
e
d'
O
R
D
E
R
B
Y
pr
ior
ity
, s
ch
e
d
ul
e
d
_f
or
e
ve
ry

5
se
co
n
ds
.
Pi
ck
s
u
p
o
n
e j
o
b
at

a
ti
m
e,
se
ts
st
at
us

= '
ru
n
ni

n
g',
e
xe
cu
te
s
it,
th
e
n
se
ts
st
at
us

= '
co
m
pl
et
e'
or
st
at
us

= '
fai
le
d'
wi
th
la
st

—
er
ro
r.

•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/i
n
d
ex
.ts
:
St
ar
t t
h
e j
o
b
w
or
ke
r
o

n
se
rv
er
st
ar
tu
p.

What stays the same: API route response times are unchanged (the route still returns 202 immediately). The populate still runs in the background. The difference is it now survives server restarts.

Dependencies: M8.1.

DB migration risk: Low. One new background process. If it fails, it logs an error and stops — the queue entry remains with status = 'failed' for inspection.

Rollback plan: Disable the job worker on startup. Revert the repopulate function to call autoPopulateAthlete directly. In-flight queue entries remain in the crawl_jobs table (harmless).

Effort: 2.5 days

Test gate:

Add a new athlete. Confirm a crawl_jobs row is created.

Restart the API server while a job is running. Confirm the job is picked up again after restart (from status = 'running' !' re-queued on startup).

Confirm the dossier populates correctly — same quality as before.

SELECT status, COUNT(*) FROM crawl_jobs GROUP BY status; — confirm no jobs are stuck in 'running' state after the server has been running for 5 minutes.

M8.3 — Recovery of interrupted jobs on server startup

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/i
n
d
ex
.ts
(s
er
ve
r s
ta
rt
u
p)
:
O
n
st
ar
tu
p,
q
u
er
y f
or
a
ny
jo

bs
w
ith
st
at
us

= '
ru
n
ni
n
g'
a
n
d
re
se
t t
h
e
m
to
st
at
us

= '
q
u
e
u
e
d'
wi
th
a
tt
e
m
pt
_c
o
u
nt

+
1.
If
at
te
m
pt
_c
o
u
nt
>
=
m
ax
_
at
te
m
pt
s,
se
t s

ta
tu
s
=
fai
le
d'.

Dependencies: M8.2.

DB migration risk: None.

Effort: 1 hour

M8.4 — Scheduled crawl triggers (basic implementation)

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
jo
b-
sc
h
e
d
ul
er
.ts
(
n
e
w
fil
e)
:
A
sc
h
e
d
ul
er
t
h
at
r
u
ns
e
ve
ry
h
o
ur
.
Q
u
er
ie

s f
or
a
thl
et
es
w
h
os
e l
as
tC
ra
wl
e
d
At

is
ol
d
er
t
h
a
n
7
d
ay
s (
or
n
ull
).
Cr
e
at
es
cr
a
wl
_j
o
bs
o
f t
yp
e
re
su
lts
_c
h
ec
k f
or
e
ac
h.
Pr
ior
ity
w
ei
g
ht
e
d
by

: active status, recent competition dates, user interest signals.
• Initially scheduled only results_ check jobs.
. Other jobs types (ranking_ings_chh

ec
k,
n
e
w
s_
sc
a
n,
s
oc
ial
_s
n
a
ps
h
ot
)
ar
e
a
d
d
e
d
in
su
bs
e
q
u
e
nt
it
er
ati
o
ns
.

What stays the same: Manual repopulate from admin still works and takes priority.

Dependencies: M8.3.

DB migration risk: None.

Effort: 1.5 days

Phase 8 test gate:

• [
]
All
r
e
p
o
p
ul
at
e
tri
g
g
er
s (
a
d
mi
n,
d
os

si
er
,
n
e
w
at
hl
et
e)
cr
e
at
e j
o
b
q
u
e
u
e
e
nt
ri
es
• [
]
S
er
ve
r r
es
ta
rt
d
o
es
n
ot
lo
se
q
u
e
u
e
d
or

in-
pr
o
gr
es
s j
o
bs
• [
]
A
d
mi
n
p
a
n
el
sh
o

w
s j
o
b
q
u
e
u
e
st
at
us
(
n
e
w
co
lu
m
n
in
Cr
a
wl
t
a
b)
• [
]
S
ch
e
d
ul
er
cr
e
at
es
jo
bs
fo
r s
tal
e
at
hl
et
es
o
n
a
re
g
ul
ar
in
te
rv
al
• [
]
N
o
d
u
pli
ca
te

jobs created for the same athlete (check uniqueness constraints)

Phase 9 — Timeline Rebuild

Extends the existing `timeline_events` table with new columns rather than replacing it. New columns are nullable initially, allowing the old pipeline to continue writing partial data while the new pipeline writes full data.

M9.1 — Add new columns to `timeline_events` (additive, nullable)`

What changes:

- `li`
`b/`
`d`
`b/`
`sr`
`c/`
`sc`
`h`
`e`
`m`
`a/`
`ti`
`m`
`eli`
`n`
`e-`
`ev`
`e`
`nt`
`s.t`
`s:`
`A`
`d`
`d`

the following nullable columnns:

- event Type text (nullable word controlled vocabulary — maps from existing category)
- event Date Precision on text d

ef
a
ult
'd
ay
,

•
ev
e
nt
D
at
e
Di
sp
la
y t
ex
t(
h
u
m
a
n-
re
a
d
a
bl
e i
m
pr
ec
is
e
d
at
es
)

•
ev
e
nt
E
n
d
D
at
e
d
at
e

• i
s
O
n
g
oi
n
g
b
o
ol
e
a
n
d
ef
a

ult
fa
ls
e
•
co
m
p
eti
tio
n
N
a
m
e
te
xt
•
co
m
p
eti
tio
n
Ti
er
t
ex
t
• r
es
ult
V
al
u
e
te
xt
• r
es
ult
N
u
m
er
ic
nu
m
er
ic
• r
es
ult
R
a
nk
in
te
g
er
•
si
g
nif
ic
a
nc
e i
nt

e
g
er
d
ef
a
ult
3
• i
s
P
b
b
o
ol
e
a
n
d
ef
a
ult
fa
ls
e
• i
s
N
ati
o
n
al
R
ec
or
d
b
o
ol
e
a
n
d
ef
a
ult
fa
ls
e
• i
s
W
or
ld
R
ec
or
d
b
o
ol
e
a
n
d
ef
a
ult
fa

Is
e
•
pr
im
ar
y
E
vi
d
e
nc
el
d
u
ui
d
re
fe
re
nc
es
e
vi
d
e
nc
e
_it
e
m
s(i
d)

—
n
ull
a
bl
e
F
K
•
su
p
er
se
d
e
d
B
y
u
ui
d
re
fe
re
nc
es
ti
m
eli
n
e
—
ev
e
nt

s(i
d)

—
n
ull
a
bl
e
se
lf-
F
K
• i
s
S
u
p
er
se
d
e
d
b
o
ol
e
a
n
d
ef
a
ult
fa
ls
e
•
cr
e
at
e
d
B
y t
ex
t
d
ef
a
ult
's
ys
te
m'

•
R
u
n
p
n
p
m
--f
ilt
er

@
w
or

ks
p
ac
e/
d
b
ru
n
d
b:
p
us
h

All new columns are nullable or have defaults. Existing rows are unaffected. The old pipeline continues to work (it doesn't write these columns, which is fine).

Dependencies: Phase 8 complete, M5.4 complete (evidence items needed for FK).

DB migration risk: Low. All additive nullable columns. PostgreSQL handles this without a table rewrite.

Rollback plan: Drop the new columns individually.

Effort: 2 hours

M9.2 — New pipeline writes timeline events with full schema

What changes:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
W
h
e
n
cr
e
ati
n
g
ti
m
eli
n
e
ev
e
nt
s,
p
o

p
ul
at
e
all
n
e
w
co
lu
m
ns

in
a
d
dit
io
n
to
t
h
e
ex
ist
in
g
o
n
es

.
M
a
p
th
e
ex
ist
in
g
ca
te
g
or
y f
iel
d
to
t
h
e
n
e
w
ev
e
nt
Ty
p
e
vo
ca
bu
lar
y.
S
et
p
ri

mary Evidence to the correct positioning given evidence in M5.3.

Dependencies: M9.1.

DB migration risk: None. Additional column values. Existing pipeline writes are unaffected.

Effort: 1 day

M9.3 — Backfill existing timeline events to the new schema

What changes:

- New script: artifacts/pipeline_scripts/

backfired
eliminated
s

•

For each existing
mimic event
with null event
type, derive event
type from category.
Set significant
= 3 (default)

ult
)
L
e
av
e
pr
im
ar
y
E
vi
d
e
nc
el
d
nu
ll
if
n
o
co
rr
es
p
o
n
di
n
g
ev
id
e
nc
e i
te
m
ca
n
b
e
m
at
ch
e
d.

Dependencies: M9.2.

UPDATE timeline_events SET event_type = null, significance = null,
DB migration risk, Low. Updates existing rows but only fills previously-null columns.

Rollback plan:
Effort: 1 day

M9.4 — Activate the correction model

What changes:

- ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o

ut
es
/a
thl
et
es
.ts
:
A
d
d
P
A
T
C
H
/a
thl
et
es
/:i
d/
ti
m
eli
n
e/
:e
ve
nt
ld
e
n
d
p
oi
nt
t
h
at
al
lo
w
s
u
p
d
ati
n
g
a
ti
m
eli
n
e
ev
e
nt
.l
ns
te
a
d
of
m
o
dif
yi

n
g
th
e
ex
ist
in
g
ro
w,
it
cr
e
at
es

a
n
e
w
ev
e
nt
a
n
d
se
ts
su
p
er
se
d
e
d
B
y
a
n
d i
s
S
u
p
er
se
d
e
d
=
tr
u
e
o
n
th
e
ol
d
o
n
e.

•
T
h
e
a
d
mi

n
p
a
n
el
(a
n
d
ev
e
nt
u
all
y t
h
e
d
os
si
er
) c
a
n
us
e
thi
s t
o
co
rr
ec
t ti
m
eli
n
e
er
ro
rs
wi
th
o
ut
lo
si
n
g
th
e
hi
st
or
y.

Dependencies: M9.3.

DB migration risk: None.

Effort: 1 day

Phase 9 test gate:

• [
]
N
e
w
p
o
p
ul
at

e
ru
ns
cr
e
at
e
ti
m
eli
n
e
ev
e
nt
s
wi
th
al
l n
e
w
co
lu
m
ns
p
o
p
ul
at
e
d
• [
] E
xi
sti
n
g
ti
m
eli
n
e
ev
e
nt
s
h
av
e
b
e
e
n
b
ac
kfi
lle
d
wi
th
e
ve
nt
_t
yp
e

•[
]
Ti
m
eli
n
e
ta
b
co
nti
nu
es
t
o l
o
a
d
co
rr
ec
tly
•[
]
C
or
re
cti
n
g
a
ti
m
eli
n
e
ev
e
nt
cr
e
at
es

a
n
e
w
ev
e
nt
a
n
d
m
ar
ks
t
h
e
ol
d
o
n
e
as
s
u
p

er
se
d
e
d
• [
]
T
h
e
ti
m
eli
n
e
vi
e
w
sh
o
w
s
o
nl
y
n
o
n-
su
p
er
se
d
e
d
ev
e
nt
s
by
d
ef
a
ult

Phase 10 — AI Layer Redesign

The most significant code change in the entire migration. The extraction pipeline is redesigned to be evidence-grounded. This is the last phase before cutover.

Risk note: Phase 10 changes the quality of data being written to the database. It must be validated thoroughly on a small set of athletes before being applied to the full roster. A feature flag (`USE_GROUNDED_EXTRACTION`) gates the new pipeline — the old pipeline remains as fallback.

M10.1 — Evidence-grounded extraction prompts

What changes:

•
ar
tif
ac
ts/
a
pi-
se
rv

er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
R
e
d
es
ig
n
th
e
G
P
T
ex
tr
ac
tio
n
pr
o
m
pt
t
o
re
q
uir
e:

- V
er
b
ati
m
q
u
ot
e
(e
vi
d
e
nc
e
_t
ex
t)
fr
o
m
th
e
P
er
pl
ex
ity

research for every extracted claim

•

Confidence capped at 49 for any item without

abbreviated quote

•

After extraction, validate

that the achievement of this is a sub-
string of the Perplexity
respective to the
text. Items that
fail this check
are assigned
confidence
"0.3" and
flagged.

g
g
e
d
as
'a
i_
es
ti
m
at
e
d'.

•
N
e
w
fe
at
ur
e
fla
g
G
R
O
U
N
D
E
D
-
E
X
T
R
A
C
T
I
O
N
=t
ru
e
in
e
nv
iro
n
m
e
nt
.
W
h
e
n
fal
se
,
ol
d
pi
p
eli
n
e
ru
ns

.

Dependencies: Phase 9 complete, Phase 5 complete (evidence_items writes in place).

DB migration risk: None. Code change only. Feature-flagged.

Rollback plan: Set GROUNDED_EXTRACTION=false in environment. Old pipeline resumes immediately without a deployment.

Effort: 3 days

Test gate:

Enable grounded extraction. Repopulate two athletes (one well-known, one obscure).

Check evidence_items: SELECT fact_value, evidence_text, confidence FROM evidence_items WHERE athlete_identity_id = \$id;

Confirm evidence_text is non-null for items with confidence > 50

Confirm confidence scores are lower for estimated items

Confirm the dossier pages still display correctly

M10.2 — Citation validation after extraction

What changes:

•

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s:
A
d
d
p
os
t-
ex
tr
ac
tio
n
va
lid
ati
o
n
st
e
p.
F
or
e
ac
h
ex
tr

ac
te
d
so
ur
ce
Ur
l,
ve
rif
y
it
a
p
p
e
ar
s
in
th
e
P
er
pl
ex
ity
ci
ta
tio
ns
li
st.
It
e
m
s
wi
th
a
n
u
nv
er
ifi
a
bl
e
so
ur
ce
Ur
l a
re
n
ot
di
sc
ar
d
e
d
bu
t
h
av
e
th
eir
c

o
nfi
d
e
nc
e
re
d
uc
e
d
to
3
0
a
n
d
co
nfi
d
e
nc
e

—
b
as
is
= '
u
nv
er
ifi
a
bl
e
_c
ita
tio
n'.

Dependencies: M10.1.
DB migration risk: None.
Effort: 1.5 days

M10.3 — Evidence-grounded summary generation

What changes:

- ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/s
u
m
m
ar
y.t

s:
B
ef
or
e
ca
lli
n
g
G
P
T,
q
u
er
y
ev
id
e
nc
e
_it
e
m
s f
or
t
h
e
at
hl
et
e
a
n
d
bu
ild

a
str
uc
tu
re
d
co
nt
ex
t
bl
oc
k (
ve
rifi
e
d
fa
ct
s,
es
ti
m
at
e
d
fa
ct
s,
o

mi
t
u
nv
er
ifi
e
d)
.
P
as
s t
hi
s
co
nt
ex
t
bl
oc
k t
o
G
P
T i
ns
te
a
d
of
m
ak
in
g
a
fr
es
h
P
er
pl
ex
ity
c
all
. T
h
e
sy
st
e
m
pr
o
m
pt
e
xp
lic
itl
y
pr
o
hi
bit
s
us
in
g

training workload.

Dependencies: M10.1, M5.4 (evidence items must be populated for summaries to have data).

DB migration risk: None.

Rollback plan: Revert summary.ts to previous implementation.

Effort: 2 days

Test gate:

Open a dossier. Generate the AI summary. Confirm it streams correctly.

Verify the summary contains only facts present in the evidence_items table for that athlete. (Check 3 specific claims manually.)

For an athlete with minimal evidence, confirm the summary says "insufficient verified data" rather than inventing claims.

M10.4 — Chat citation system

What changes:

-

artifacts/analysis/outputs/chat.ts: Before each chat request, populate, prefetch, and

Le
vi
d
e
nc
e
_it
e
m
s f
or
t
h
e
rel
ev
a
nt
a
thl
et
es
.
P
as
s t
h
e
m
as

a
str
uc
tu
re
d
co
nt
ex
t
bl
oc
k.
In
str
uc
t t
h
e
m
o
d
el
to
a
p
p
e
n
d
[e
v-
U
UI
D]
ci
ta
tio

ns
fo
r
ev
er
y
cl
ai
m.
P
ar
se
t
h
e
re
sp
o
ns
e
to
e
xtr
ac
t c
ita
tio
n
ID
s.
R
et
ur
n
cit
ati
o
ns
al
o
n
gs
id
e
th
e
re
sp
o
ns
e.
•
ar
tif
ac
ts/
w
e
b/
sr
c/
p
a
g
es
/C
h
at
P

a
g
e.t
sx
:
R
e
n
d
er
ci
ta
tio
n
ID
s
as
fo
ot
n
ot
e i
n
di
ca
to
rs.
C
lic
ki
n
g
a
cit
ati
o
n
o
p
e
ns

a
p
a
n
el
sh
o
wi
n
g
th
e
ev
id
e
nc
e i
te
m
(s
o
ur
ce
U
R
L,
r

et
ri
ev
al
d
at
e,
co
nfi
d
e
nc
e,
ev
id
e
nc
e
te
xt)

.

Dependencies: M10.3.

DB migration risk: None.

Effort: 3 days

Phase 10 test gate:

• [
]

Gr

o

u

n

d

e

d

ex

tr

ac

tio

n

pr

o

d

uc

es

e

vi

d

e

nc

e

_t

ex

t f

or

>

8

0

%

o

f

ex

tr

ac

te

d i

nt

ell

ig
e
nc
e i
te
m
s
• [
]
C
o
nfi
d
e
nc
e
sc
or
es
c
or
re
ctl
y r
efl
ec
ts
o
ur
ce
q
u
ali
ty
• [
]
Al
s
u
m
m
ar
y
d
o
es
n
ot
c
o
nt
ai
n
cl
ai
m
s
a
bs
e
nt
fr
o
m
ev
id
e
nc
e

_it
e
m
s
• [
]
C
h
at
r
es
p
o
ns
es
in
cl
u
d
e
cit
ati
o
n
ID
s f
or
cl
ai
m
s
• [
]
Cl
ic
ki
n
g
a
cit
ati
o
n
sh
o
w
s t
h
e
so
ur
ce
e
vi
d
e
nc
e
• [
]
Ol
d
pi
p
eli
n
e
sti
ll

Phase 11 — Cutover and Cleanup

Only reached after Phase 10 has run in production for 30 days with no data quality regressions. At this point, the new system has proven itself and the old structures can be deprecated.

M11.1 — Frontend reads from new schema

What changes:

The dossier page progressively migrates to reading from the new tables:

- I nt ell ig e nc e ta b l e r e a ds fr o m ev id e nc e _it e m s i ns te a

d
of
in
tel
lig
e
nc
e
_it
e
m
s

•
Ti
m
eli
n
e
ta
b
!' r
e
a
ds
fr
o
m
re
d
es
ig
n
e
d
ti
m
eli
n
e

—
ev
e
nt
s (
n
e
w
co
lu
m
ns
)

•
S
oc
ial
M
e
di
a
ta
b
!' r
e
a
ds
fr
o
m

so
ci
al
_s
n
a
ps
h
ot
s i
ns
te
a
d
of
a
thl
et
es
.in
st
a
gr
a
m
F
oll
o
w
er
s
et
c.
•
C
o
nt
ac
ts
ta
b
! ' r
e
a
ds
fr
o
m
ev
id
e
nc
e
_it
e
m
s
w
h
er
e
fa
ct
_t
yp
e
IN
(
co

ac
h',
'a
g
e
nt'
,
sp
o
ns
or
_c
o
nt
ac
t')
in
st
e
a
d
of
c
o
nt
ac
ts
•
C
o
m
p
eti
tio
ns
t
a
b
! ' r
e
a
ds
fr
o
m
ti
m
eli
n
e
—
ev
e
nt
s
w
h
er
e
ev
e
nt
_t
yp
e '
co
m
p

eti
tio
n
_r
es
ult
'i
ns
te
a
d
of
c
o
m
p
eti
tio
ns

What stays the same during migration: The old backend routes remain. A feature flag per section controls which data source is active. Sections switch over one at a time.

Dependencies: Phase 10 complete and stable for 30 days.

DB migration risk: Low. No schema changes. Read queries only.

Rollback plan: Revert feature flags to point to old tables.

Effort: 1 week

M11.2 — Deprecate old tables (read-only for 30 days)

What changes:

-

R
ev
ok
e
IN
S
E
R
T
a
n
d
U
P
D
A
T
E
pr
ivi
le
g
es
o
n i
nt
ell
ig
e
nc
e
_it
e
m
s,
co

nt
ac
ts,
c
o
m
p
eti
tio
ns
,
a
n
d
th
e
so
ci
al
co
lu
m
ns
o
n
at
hl
et
es
.T
h
e
ta
bl
es
r
e
m
ai
n
re
a
d
a
bl
e.

• A
ny
c
o
d
e
p
at
hs
st
ill
wr
iti
n
g
to
t
h
es
e
ta
bl

es
w
ill
th
ro
w
a
n
er
ro
r
a
n
d
m
us
t
b
e
fix
e
d
b
ef
or
e
thi
s
mi
le
st
o
n
e.
•
M
o
nit
or
fo
r
a
ny
e
rr
or
s f
or
3
0
d
ay
s.

Dependencies: M11.1 complete and stable.

DB migration risk: Medium. This is the irreversible step. Ensure all write paths have been migrated first.

Rollback plan: Re-grant write privileges. The tables still contain data.

Effort: 1 day

M11.3 — Drop deprecated tables and old columns

What changes:

After 30 days of read-only operation with zero errors:

•
D
R
O
P

T
A
B
L
E
int
ell
ig
e
nc
e
_it
e
m
s;
•
D
R
O
P
T
A
B
L
E
co
nt
ac
ts;
•
D
R
O
P
T
A
B
L
E
co
m
p
eti
tio
ns
;
•
A
LT
E
R
T
A
B
L
E
at
hl
et
es
D
R
O
P
C
O
L
U
M

N
in
st
a
gr
a
m

—
h
a
n
dl
e,
in
st
a
gr
a
m
_f
oll
o
w
er
s,
in
st
a
gr
a
m

—
e
n
g
a
g
e
m
e
nt
, t
wi
tt
er

—
h
a
n
dl
e,
tw
itt
er
_f
oll
o
w
er
s,
tik
to
k_
h
a
n
dl
e,
tik

to
k_
fol
lo
w
er
s,
fol
lo
w
er
_
gr
o
wt
h
_
3
0
d,
a
vg
_
e
n
g
a
g
e
m
e
nt
,
ai
_s
u
m
m
ar
y,
ai
_s
u
m
m
ar
y_
g
e
n
er
at
e
d
_
at
;
•
R
e
m
ov
e
th
e
co
rr
es
p

[illegible]

Dependencies: M13.2) complete for 30 days. 7 All phases depend on % % % Phase 1

Phase 10 (M10.1-M10.4) depends on: Phase 9

Efforts 13 and 14 (most verification, no blocking)

Dependency Graph Risk Register

Risk	Likelihood	Impact	Mitigation
Perplexity returns insufficient data for a specific athlete	High	Medium	Pipeline aborts cleanly (M1.3); athlete marked population_status = 'failed'; admin notified
DB migration fails mid-run (e.g. network drop)	Low	Low	All migrations are additive with idempotent scripts; re-run safely
GPT returns malformed JSON in grounded extraction	Medium	Low	Existing JSON parse error handling retained; evidence item not written; warning logged
Backfill script creates wrong evidence items	Medium	Medium	Backfill items have confidence = 35, verification_status = 'unverified'; clearly labelled; can be truncated and re-run
Duplicate athlete_identities created before fingerprint index	Low	Low	Idempotent design; identity merge function handles post-hoc cleanup
Phase 10 degrades data quality vs current pipeline	Medium	High	Feature flag; old pipeline available; 30-athlete test run before enabling broadly
Old tables dropped before new tables fully validated	Low	High	30-day read-only period; mandatory backup before any DROP

Twitter API rate limit hit during
social snapshot

Medium

Low

What Existing Athletes Experience at Each Phase

Phase	Effect on existing athlete dossiers
Phase 1	Better data quality on next repopulate. Nothing visible changes immediately.
Phase 2	Faster page loads (queries use indexes). Nothing visible changes.
Phase 3	Source registry populated. Intelligence items begin showing source tiers on hover.
Phase 4	Progress labels during populate ("Researching...", "Extracting...") instead of silent spinner.
Phase 5	Evidence layer running silently in background. No visible change.
Phase 6	New athletes get permanent identities. Existing athletes backfilled. No visible change.
Phase 7	Social data shows verified/estimated labels and "as of [date]" timestamp.
Phase 8	Populate jobs survive server restarts. No visible change to users.
Phase 9	Timeline events show competition tier, significance, and correction history.
Phase 10	Intelligence items show higher confidence for verified claims. Summaries and chat are more accurate.
Phase 11	UI reads from new schema. If migration is correct, zero visible difference.

End of roadmap. No code has been written. Awaiting approval to begin Phase 1.